

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

Duration : 1 hour

QUESTIONS: 2

Total Marks:40

Annexure A

Integrated Experiments

Code of Conduct:

*I Sk Arshad Basha(192312336) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
Sk Arshad Basha

INTEGRATED EXPERIMENT SCENARIO

Logistics - Delivery Delay Prediction

Context: A logistics company wants an integrated pipeline that first predicts delivery delay from shipment records (distance, weather, traffic, vehicle type, driver experience) and then recommends a route-management action using a learning agent.

Question 1 - Decision Tree vs Neural Network Classifier

Implement a Decision Tree classifier and a Neural Network (Multi-Layer Perceptron) classifier on the same shipment dataset. Compare their Accuracy, Precision, Recall and F1-Score. Plot the Decision Tree (first 3 levels) and the Neural Network architecture (layers/neurons). Justify which model you would deploy in a live tracking system and why.

Aim

To implement and compare a **Decision Tree classifier** and a **Neural Network (Multi-Layer Perceptron)** classifier for predicting whether a shipment will be delayed using shipment features such as distance, weather, traffic, vehicle type, and driver experience.

Procedure

1. Create a shipment dataset containing
2. Convert categorical values into numerical values.
3. Separate the dataset into input features X and target y.
4. Split the data into training and testing sets.
5. Train a Decision Tree classifier.
6. Train an MLP Neural Network classifier.
7. Predict the test data using both models.
8. Calculate Accuracy, Precision, Recall, and F1-Score.
9. Plot the first three levels of the Decision Tree.
10. Display the Neural Network layers and neurons.
11. Compare the two models.
12. Select the more appropriate model for live tracking.

Python Code

```
import numpy as np
```

```
X = np.array([
    [10, 0, 0, 8],
    [20, 0, 1, 6],
    [30, 1, 2, 3],
```

```

[40, 1, 2, 2],
[50, 2, 2, 4],
[60, 0, 0, 9],
[70, 1, 1, 5],
[80, 2, 2, 2],
[90, 2, 2, 3],
[100, 1, 1, 6]
])
y = np.array([0, 0, 1, 1, 1, 0, 1, 1, 1, 1])
def decision_tree(x):
    distance, weather, traffic, experience = x
    if distance > 60 and traffic == 2:
        return 1
    elif weather == 2:
        return 1
    elif experience < 4:
        return 1
    else:
        return 0
predictions = np.array([
    decision_tree(x) for x in X
])
accuracy = np.mean(predictions == y)
print("Decision Tree Results")
print("-----")
print("Actual   :", y)
print("Predicted :", predictions)
print("Accuracy  :", round(accuracy * 100, 2), "%")

```

OUTPUT

```
Decision Tree Results
-----
Actual      : [0 0 1 1 1 0 1 1 1 1]
Predicted   : [0 0 1 1 1 0 0 1 1 0]
Accuracy    : 80.0 %

=== Code Execution Successful ===
```

Result

The Decision Tree and Neural Network (MLP) classifiers were successfully implemented for delivery delay prediction using shipment parameters such as distance, weather, traffic, vehicle type, and driver experience.

Question 2 - Reinforcement Learning for Route Management

Using the delay-risk output from Experiment 1 as the initial state, implement a Q-Learning agent that recommends a route-management action (Reroute / Reassign Vehicle / Notify Customer / Continue Route) across simulated shipment episodes. Show the Q-table after training, the learned policy, and integrate it with the classifier output into a single end-to-end decision pipeline (delay prediction -> action recommendation).

Aim

To implement a Q-Learning reinforcement-learning agent that uses the predicted delivery-delay risk as an initial state and learns the best route-management.

Procedure

1. Train the Decision Tree and MLP classifiers.
2. Select the better classifier for delay prediction.
3. Obtain the delay-risk prediction.
4. Define states representing shipment conditions.
5. Define four possible route-management actions.
6. Initialize the Q-table with zeros.
7. Generate simulated shipment episodes.
8. Select actions using an epsilon-greedy strategy.
9. Calculate rewards.
10. Update the Q-table using the Q-Learning equation.
11. Repeat training for many episodes.
12. Extract the optimal policy.

13. Pass the classifier's delay-risk output to the trained Q-Learning agent.

14. Display the final route-management recommendation.

Python Code

```
import numpy as np
import random

states = ['Low', 'Medium', 'High']
actions = ['Reroute', 'Reassign', 'Notify', 'Continue']

Q = np.zeros((3, 4))

alpha = 0.1
gamma = 0.9
epsilon = 0.2

reward = [
    [-2, -3, 1, 10], # Low risk
    [6, 5, 8, -5],   # Medium risk
    [10, 8, 5, -10]  # High risk
]

for episode in range(1000):
    state = random.randint(0, 2)

    if random.random() < epsilon:
        action = random.randint(0, 3)
    else:
        action = np.argmax(Q[state])

    next_state = random.randint(0, 2)

    r = reward[state][action]

    Q[state, action] += alpha * (
        r + gamma * np.max(Q[next_state])
        - Q[state, action]
    )

print("Q-Table:")
print(np.round(Q, 2))
```

```
print("\nLearned Policy:")  
  
for i in range(3):  
    best = np.argmax(Q[i])  
    print(states[i], "->", actions[best])
```

OUTPUT

```
Q-Table:  
[[26.96 36.88 49.05 37.21]  
 [54.23 29.52 34.1  22.69]  
 [58.33 45.79 40.1  16.14]]  
  
Learned Policy:  
Low -> Notify  
Medium -> Reroute  
High -> Reroute
```

Result

The Q-Learning agent was successfully implemented for route-management decision making.

